

subject:- Computer Vision (S1-25_AIMLCZG525) Group- 95 1.Name:-ADI SATHWIK; BITS ID:-2024AB05213; contribution:-100% 2.Name:-AKASH PAREEK; BITS ID:-2023AC05551; contribution:-100% 3.Name:-JAGADI SHRUTI CHANABASAPPA; BITS ID:-2024AA05181; contribution:-100% 4.Name:-MAYANK NAYYAR; BITS ID:-2024AA05716; contribution:-100% 5.Name:-R.DEEPIKA; BITS ID:-2024AA05339; contribution:-100%

Problem Statement 1: Cross-Modal Attention Framework for Robust NO₂ Prediction. PART 1: DATA PREPROCESSING & AUGMENTATION

1.1 Ingest TROPOMI Level-2 NO₂ Products • Tropospheric NO₂ column density • Quality assurance (QA) filtering

PART 1.1 – Data Ingestion

```
import xarray as xr
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Load Sentinel-5P Level-2 dataset
dataset = xr.open_dataset("S5P_N02_L2.nc")

# Extract variables
no2 = dataset["tropospheric_N02_column_number_density"]
qa = dataset["qa_value"]
cloud = dataset["cloud_fraction"]

print("Dataset loaded successfully")
print("N02 Shape:", no2.shape)

# Apply QA threshold filtering
no2_qa_filtered = no2.where(qa > 0.75)

print("After QA filtering:", no2_qa_filtered.shape)

Dataset loaded successfully
N02 Shape: (10, 50, 50)
After QA filtering: (10, 50, 50)
```

1.2 Cloud Masking and QA-Based Filtering

```
# Cloud masking (remove cloud_fraction > 0.3)
no2_filtered = no2_qa_filtered.where(cloud < 0.3)

print("After cloud masking:", no2_filtered.shape)

After cloud masking: (10, 50, 50)
```

1.3 Regridding to Uniform Spatial Resolution

```
# Regrid (coarsen resolution)
no2_regrid = no2_filtered.coarsen(latitude=2, longitude=2,
boundary="trim").mean()

print("Regridded shape:", no2_regrid.shape)

Regridded shape: (10, 25, 25)
```

1.4 Temporal Aggregation (Daily / Weekly Averages)

```
# Daily Aggregation
# Daily average
daily_no2 = no2_regrid.resample(time="1D").mean()

print("Daily Aggregated Shape:", daily_no2.shape)

# Weekly Aggregation
# Weekly Aggregation
weekly_no2 = no2_regrid.resample(time="1W").mean()

print("Weekly Aggregated Data:")
print(weekly_no2)

print("\nWeekly Aggregated Shape:", weekly_no2.shape)
print("Number of Weeks:", weekly_no2.sizes["time"])

Daily Aggregated Shape: (10, 25, 25)
Weekly Aggregated Data:
<xarray.DataArray 'tropospheric_N02_column_number_density' (time: 3,
                                                                latitude:
25,
                                                                longitude:
25)> Size: 15kB
array([[[ nan, 6.52302986e+15, nan, ...,
3.53648505e+15, 5.29612028e+15, nan],
[ nan, 5.23225370e+15, 4.83871429e+15, ...,
nan, 3.76304929e+15, 5.52194157e+15],
[ nan, 6.05380205e+15, 4.51576593e+15, ...,
nan, 5.89959988e+15, nan],
...,
[4.87245088e+15, 3.54882431e+15, nan, ...,
5.52251411e+15, 5.90999841e+15, nan],
[5.83847117e+15, 5.14979903e+15, nan, ...,
nan, 4.52116254e+15, 4.90762734e+15],
[5.17682087e+15, 4.36877996e+15, 6.41385360e+15, ...,
4.68457451e+15, 5.29422413e+15, 4.21623433e+15]],
[[[5.08813641e+15, 4.64331904e+15, 4.24198869e+15, ...,
5.14237030e+15, 4.66324391e+15, 5.36079432e+15],
[3.96547942e+15, 4.39427342e+15, 4.62011045e+15, ...,
```

```

4.92391418e+15, 5.00777386e+15, 5.55381196e+15],
[5.07603813e+15, 5.23378434e+15, 5.58684056e+15, ...,
5.31201894e+15, 4.27574269e+15, 5.59078598e+15],
...
5.58256791e+15, 5.10590500e+15, 4.91633468e+15],
[4.63746765e+15, 4.79239216e+15, 5.32922351e+15, ...,
4.73573107e+15, 4.86653020e+15, 5.23811433e+15],
[5.23322073e+15, 4.89006245e+15, 5.00076048e+15, ...,
5.43153712e+15, 5.13426328e+15, 5.38286273e+15]],

[[[4.10603976e+15, 5.78737514e+15, 4.57675365e+15, ...,
5.12348031e+15, 5.38237530e+15, 4.49071804e+15],
[5.44005066e+15, 4.71739631e+15, 5.41487490e+15, ...,
5.02872869e+15, 3.84071760e+15, 6.37590842e+15],
[5.29353117e+15, 4.90387065e+15, 5.92222768e+15, ...,
nan, 5.87409340e+15, 5.13603870e+15],
...,
[4.07029601e+15, 5.87599038e+15, 4.72100497e+15, ...,
5.06246011e+15, 5.29735254e+15, nan],
[4.55891075e+15, 5.71059486e+15, 4.18273708e+15, ...,
nan, nan, 4.11165726e+15],
[5.50087917e+15, nan, nan, nan, ...,
3.30007572e+15, 4.76327823e+15, 6.76317493e+15]]],
shape=(3, 25, 25))
Coordinates:
* time          (time) datetime64[ns] 24B 2023-01-01 2023-01-08 2023-
01-15
* latitude      (latitude) float64 200B 10.1 10.51 10.92 ... 19.08
19.49 19.9
* longitude     (longitude) float64 200B 70.1 70.51 70.92 ... 79.08
79.49 79.9

Weekly Aggregated Shape: (3, 25, 25)
Number of Weeks: 3

```

1.5 Normalization and Outlier Removal

```

# =====
# Convert to NumPy
# =====
no2_np = daily_no2.values

print("Original N02 Data Shape:", no2_np.shape)
print("Original Min:", np.nanmin(no2_np))
print("Original Max:", np.nanmax(no2_np))
print("Original Mean:", np.nanmean(no2_np))
print("Original Std:", np.nanstd(no2_np))

```

```

# =====
# Normalize (Z-score)
# =====
mean = np.nanmean(no2_np)
std = np.nanstd(no2_np)

no2_norm = (no2_np - mean) / std

print("\nAfter Normalization:")
print("Normalized Min:", np.nanmin(no2_norm))
print("Normalized Max:", np.nanmax(no2_norm))
print("Normalized Mean (should be ~0):", np.nanmean(no2_norm))
print("Normalized Std (should be ~1):", np.nanstd(no2_norm))

# =====
# Outlier Removal (Clipping)
# =====
no2_norm = np.clip(no2_norm, -3, 3)

print("\nAfter Outlier Removal (Clipping to [-3, 3]):")
print("Clipped Min:", np.nanmin(no2_norm))
print("Clipped Max:", np.nanmax(no2_norm))

# =====
# Show Sample Values
# =====
print("\nSample Normalized Values (first 5 elements):")
print(no2_norm.flatten()[:5])

Original NO2 Data Shape: (10, 25, 25)
Original Min: 1344913484895894.0
Original Max: 8942331010806686.0
Original Mean: 5004353708630748.0
Original Std: 910831713626085.4

After Normalization:
Normalized Min: -4.017690830248283
Normalized Max: 4.323496034737935
Normalized Mean (should be ~0): 3.7984465123869363e-16
Normalized Std (should be ~1): 1.0

After Outlier Removal (Clipping to [-3, 3]):
Clipped Min: -3.0
Clipped Max: 3.0

Sample Normalized Values (first 5 elements):
[      nan 1.66735098      nan 0.74085184 0.59089547]

```

1.6 Integrate Auxiliary Data Assignment requires: Meteorological variables Land-use / population density Temporal indicators

```
# Simulated auxiliary data (replace with real if available)

num_samples = no2_norm.shape[0]

temperature = np.random.normal(25, 5, num_samples)
wind_speed = np.random.normal(5, 2, num_samples)
humidity = np.random.uniform(40, 90, num_samples)
population_density = np.random.uniform(100, 1000, num_samples)

# Temporal indicators
dates = pd.to_datetime(daily_no2.time.values)
day_of_week = dates.dayofweek
season = dates.month

# Combine into Feature Matrix
aux_data = np.stack([
    temperature,
    wind_speed,
    humidity,
    population_density,
    day_of_week
], axis=1)

print("Auxiliary Data Shape:", aux_data.shape)

Auxiliary Data Shape: (10, 5)
```

1.7 Data Augmentation

```
# Temporal Jittering
no2_aug = np.roll(no2_norm, shift=1, axis=0)

# Noise Injection
noise = np.random.normal(0, 0.01, no2_aug.shape)
no2_aug = no2_aug + noise

# Spatial Smoothing
from scipy.ndimage import gaussian_filter

no2_aug = gaussian_filter(no2_aug, sigma=1)

# Spatial Masking
mask = np.random.rand(*no2_aug.shape) > 0.1
no2_aug = no2_aug * mask

# Convert to PyTorch Tensors
import torch
```

```

data_tensor = torch.tensor(no2_aug, dtype=torch.float32)

# Add channel dimension
data_tensor = data_tensor.unsqueeze(1)

aux_tensor = torch.tensor(aux_data, dtype=torch.float32)

target = torch.tensor(no2_norm.reshape(num_samples, -1)[: ,0:1],
dtype=torch.float32)

print("Satellite Tensor Shape:", data_tensor.shape)
print("Aux Tensor Shape:", aux_tensor.shape)
print("Target Shape:", target.shape)

Satellite Tensor Shape: torch.Size([10, 1, 25, 25])
Aux Tensor Shape: torch.Size([10, 5])
Target Shape: torch.Size([10, 1])

# PART 1.1 – Data Ingestion

import xarray as xr
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
dataset = xr.open_dataset("S5P_N02_L2.nc")

# Extract required variables
no2 = dataset["tropospheric_N02_column_number_density"]
qa = dataset["qa_value"]
cloud = dataset["cloud_fraction"]

print("Dataset Loaded Successfully")
print("N02 shape:", no2.shape)

Dataset Loaded Successfully
N02 shape: (10, 50, 50)

```

1.2 Cloud Masking and QA-Based Filtering

```

# PART 1.2 – QA Filtering + Cloud Masking

no2_filtered = no2.where((qa > 0.75) & (cloud < 0.3), drop=True)

print("Filtered N02 shape:", no2_filtered.shape)

Filtered N02 shape: (10, 50, 50)

```

1.3 Regridding to Uniform Spatial Resolution

PART 1.3 – Regridding

```
no2_regrid = no2_filtered.coarsen(latitude=2, longitude=2,  
boundary="trim").mean()
```

```
print("Regridded Shape:", no2_regrid.shape)
```

Regridded Shape: (10, 25, 25)

1.4 Temporal Aggregation (Daily Mean)

PART 1.4 – Temporal Aggregation

```
daily_mean = no2_regrid.resample(time="1D").mean()
```

```
print("Daily Aggregated Shape:", daily_mean.shape)
```

Daily Aggregated Shape: (10, 25, 25)

1.5 Normalization and Outlier Removal

PART 1.5 – Normalization

```
print("===== NORMALIZATION STARTED =====\n")
```

```
# -----
```

```
# Step 1: Convert to NumPy
```

```
# -----
```

```
no2_np = daily_mean.values
```

```
print("Original Data Shape:", no2_np.shape)
```

```
print("Original Min:", np.nanmin(no2_np))
```

```
print("Original Max:", np.nanmax(no2_np))
```

```
print("Original Mean:", np.nanmean(no2_np))
```

```
print("Original Std:", np.nanstd(no2_np))
```

```
# -----
```

```
# Step 2: Compute Mean & Std
```

```
# -----
```

```
mean = np.nanmean(no2_np)
```

```
std = np.nanstd(no2_np)
```

```
print("\nComputed Mean:", mean)
```

```
print("Computed Std Dev:", std)
```

```
# -----
```

```
# Step 3: Z-score Normalization
```

```
# -----
```

```

no2_norm = (no2_np - mean) / std

print("\nAfter Z-score Normalization:")
print("Min:", np.nanmin(no2_norm))
print("Max:", np.nanmax(no2_norm))
print("Mean (should be ~0):", np.nanmean(no2_norm))
print("Std (should be ~1):", np.nanstd(no2_norm))

# -----
# Step 4: Outlier Clipping
# -----
no2_norm = np.clip(no2_norm, -3, 3)

print("\nAfter Clipping to [-3, 3]:")
print("Min:", np.nanmin(no2_norm))
print("Max:", np.nanmax(no2_norm))

# -----
# Step 5: Show Sample Values
# -----
print("\nSample Normalized Values (first 5):")
print(no2_norm.flatten()[:5])

print("\n===== NORMALIZATION COMPLETED =====")

===== NORMALIZATION STARTED =====

Original Data Shape: (10, 25, 25)
Original Min: 1344913484895894.0
Original Max: 8942331010806686.0
Original Mean: 5004353708630748.0
Original Std: 910831713626085.4

Computed Mean: 5004353708630748.0
Computed Std Dev: 910831713626085.4

After Z-score Normalization:
Min: -4.017690830248283
Max: 4.323496034737935
Mean (should be ~0): 3.7984465123869363e-16
Std (should be ~1): 1.0

After Clipping to [-3, 3]:
Min: -3.0
Max: 3.0

Sample Normalized Values (first 5):
[      nan 1.66735098      nan 0.74085184 0.59089547]

```



```
===== NORMALIZATION COMPLETED =====
```

1.6 Integrate Auxiliary Data

```
# PART 1.6 – Dummy Auxiliary Data
```

```
import torch
```

```
num_samples = no2_norm.shape[0]
```

```
aux_dim = 5
```

```
aux_data = torch.randn(num_samples, aux_dim)
```

```
target = torch.randn(num_samples, 1)
```

```
print("Auxiliary Data Shape:", aux_data.shape)
```

```
Auxiliary Data Shape: torch.Size([10, 5])
```

1.7 Data Augmentation • Noise Injection • Temporal Jittering • Spatial Masking

```
# PART 1.7 – Data Augmentation
```

```
print("===== DATA AUGMENTATION STARTED =====\n")
```

```
# -----
```

```
# Step 1: Original Data Summary
```

```
# -----
```

```
print("Original NO2 Data Stats:")
```

```
print("Shape:", no2_norm.shape)
```

```
print("Min:", np.nanmin(no2_norm))
```

```
print("Max:", np.nanmax(no2_norm))
```

```
print("Mean:", np.nanmean(no2_norm))
```

```
print("Std:", np.nanstd(no2_norm))
```

```
# -----
```

```
# Step 2: Noise Injection
```

```
# -----
```

```
noise = np.random.normal(0, 0.01, no2_norm.shape)
```

```
print("\nNoise Stats:")
```

```
print("Noise Mean:", np.mean(noise))
```

```
print("Noise Std:", np.std(noise))
```

```
no2_aug = no2_norm + noise
```

```
print("\nAfter Noise Injection:")
```

```
print("Min:", np.nanmin(no2_aug))
```

```
print("Max:", np.nanmax(no2_aug))
```

```

# -----
# Step 3: Temporal Jittering
# -----
no2_aug = np.roll(no2_aug, shift=1, axis=0)

print("\nTemporal Jittering Applied (Shift=1 along time axis)")
print("Shape after roll:", no2_aug.shape)

# -----
# Step 4: Spatial Masking
# -----
mask = np.random.rand(*no2_aug.shape) > 0.1
no2_aug = no2_aug * mask

masked_percentage = 100 * (1 - np.mean(mask))

print("\nSpatial Masking Applied")
print(f"Percentage of Masked Pixels: {masked_percentage:.2f}%")

print("\nAfter Masking:")
print("Min:", np.nanmin(no2_aug))
print("Max:", np.nanmax(no2_aug))

# -----
# Step 5: Show Sample Values
# -----
print("\nSample Augmented Values (first 5 elements):")
print(no2_aug.flatten()[:5])

print("\n===== DATA AUGMENTATION COMPLETED =====")

===== DATA AUGMENTATION STARTED =====

Original N02 Data Stats:
Shape: (10, 25, 25)
Min: -3.0
Max: 3.0
Mean: -0.0002682256948304918
Std: 0.9928297516426646

Noise Stats:
Noise Mean: -0.0001880586731000364
Noise Std: 0.00998212047263086

After Noise Injection:
Min: -3.020443072948654
Max: 3.016092297495454

```

Temporal Jittering Applied (Shift=1 along time axis)
Shape after roll: (10, 25, 25)

Spatial Masking Applied
Percentage of Masked Pixels: 9.98%

After Masking:
Min: -3.020443072948654
Max: 3.016092297495454

Sample Augmented Values (first 5 elements):
[nan 0.86211845 -0.21789418 nan nan]

===== DATA AUGMENTATION COMPLETED =====

PART 2: CROSS-MODAL FEATURE EXTRACTION & FUSION

2.1 Multi-Branch Architecture Design

```
import torch
import torch.nn as nn

# =====
# TROPOMI Branch - CNN
# =====
class SatelliteBranch(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.relu1 = nn.ReLU()
        self.pool = nn.MaxPool2d(2)

        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.relu2 = nn.ReLU()

        self.avgpool = nn.AdaptiveAvgPool2d((1,1))

    def forward(self, x):
        print("Input Shape:", x.shape)

        x = self.conv1(x)
        print("After Conv1:", x.shape)

        x = self.relu1(x)
        x = self.pool(x)
        print("After MaxPool:", x.shape)

        x = self.conv2(x)
        print("After Conv2:", x.shape)
```

```

        x = self.relu2(x)
        x = self.avgpool(x)
        print("After AdaptiveAvgPool:", x.shape)

        x = x.view(x.size(0), -1)
        print("Final Flattened Output:", x.shape)

    return x

# =====
# Create Model
# =====
sat_model = SatelliteBranch()

print("\nSatellite Branch Architecture:\n")
print(sat_model)

# =====
# Test with Dummy Input
# =====
sample_input = torch.randn(4, 1, 50, 50)
sat_output = sat_model(sample_input)

print("\nSatellite Feature Output (first 2 rows):\n", sat_output[:2])

# =====
# Auxiliary Data Branch – MLP
# =====
class AuxiliaryBranch(nn.Module):
    def __init__(self, aux_dim):
        super().__init__()
        self.fc1 = nn.Linear(aux_dim, 64)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(64, 32)

    def forward(self, x):
        print("Aux Input Shape:", x.shape)

        x = self.fc1(x)
        print("After FC1:", x.shape)

        x = self.relu(x)

        x = self.fc2(x)
        print("Final Auxiliary Feature Shape:", x.shape)

        return x

# =====

```

```

# Create Model
# =====
aux_dim = 5
aux_model = AuxiliaryBranch(aux_dim)

print("\nAuxiliary Branch Architecture:\n")
print(aux_model)

# =====
# Test with Dummy Input
# =====
sample_aux = torch.randn(4, aux_dim)
aux_output = aux_model(sample_aux)

print("\nAuxiliary Feature Output (first 2 rows):\n", aux_output[:2])

```

Satellite Branch Architecture:

```

SatelliteBranch(
  (conv1): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  (relu1): ReLU()
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
  (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  (relu2): ReLU()
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
)

```

```

Input Shape: torch.Size([4, 1, 50, 50])
After Conv1: torch.Size([4, 16, 50, 50])
After MaxPool: torch.Size([4, 16, 25, 25])
After Conv2: torch.Size([4, 32, 25, 25])
After AdaptiveAvgPool: torch.Size([4, 32, 1, 1])
Final Flattened Output: torch.Size([4, 32])

```

```

Satellite Feature Output (first 2 rows):
tensor([[1.3245e-02, 5.6217e-01, 4.7465e-01, 1.1039e-01, 1.4265e-04,
1.4491e-01,
        6.3317e-03, 8.7180e-01, 3.9701e-02, 6.0761e-01, 1.0864e-01,
3.2207e-02,
        2.4926e-02, 6.8648e-02, 2.4724e-02, 1.8038e-01, 8.0559e-03,
9.8307e-03,
        7.1489e-02, 3.3741e-01, 0.0000e+00, 3.0292e-01, 3.4747e-04,
1.5045e-01,
        1.3557e-01, 4.5969e-02, 1.8582e-01, 8.4094e-02, 2.2205e-03,
1.3047e-01,
        3.0769e-01, 2.1795e-01],
        [9.6759e-03, 5.5464e-01, 4.6308e-01, 1.1821e-01, 0.0000e+00,

```

```
1.5779e-01,
      5.3238e-03, 8.7187e-01, 3.0994e-02, 6.1123e-01, 1.1037e-01,
3.3817e-02,
      2.0883e-02, 7.7883e-02, 2.0008e-02, 2.0327e-01, 1.1497e-02,
1.1035e-02,
      7.5132e-02, 3.4373e-01, 0.0000e+00, 3.0404e-01, 8.1487e-05,
1.4423e-01,
      1.2874e-01, 4.1884e-02, 1.8999e-01, 6.9413e-02, 2.9939e-03,
1.2440e-01,
      3.1314e-01, 2.2734e-01]], grad_fn=<SliceBackward0>)
```

Auxiliary Branch Architecture:

```
AuxiliaryBranch(
  (fc1): Linear(in_features=5, out_features=64, bias=True)
  (relu): ReLU()
  (fc2): Linear(in_features=64, out_features=32, bias=True)
)
```

Aux Input Shape: torch.Size([4, 5])

After FC1: torch.Size([4, 64])

Final Auxiliary Feature Shape: torch.Size([4, 32])

Auxiliary Feature Output (first 2 rows):

```
tensor([[ -4.1334e-01,  8.3466e-02,  1.6464e-02,  4.2902e-02, -
 1.9265e-01,
         2.9882e-02, -1.8922e-01, -1.3468e-02, -3.7268e-01, -1.2313e-
01,
         3.0269e-01,  3.2818e-01,  2.9475e-01, -3.8993e-02,  1.0168e-
01,
         1.7618e-01,  4.0015e-01,  7.9005e-02, -1.6065e-01,  4.4208e-
01,
        -7.1641e-01, -4.9366e-01,  7.0139e-01, -6.6654e-02,  2.8211e-
01,
         3.3638e-01,  1.4328e-01,  1.2863e-01,  1.1394e-02,  2.6428e-
01,
        -5.0541e-01, -2.9342e-01],
        [-2.6571e-01, -9.3081e-02, -3.7068e-02, -4.5120e-01, -7.7428e-
02,
        -5.6060e-01, -1.0043e-01, -5.8584e-02, -1.0914e-03, -1.8918e-
04,
         3.1287e-01, -3.3038e-02, -5.4763e-02,  3.1165e-02, -2.4690e-
02,
         2.5764e-01,  2.9277e-01,  4.2671e-01, -2.8676e-02,  2.0861e-
01,
        -6.7746e-01, -7.4307e-01,  2.3883e-01, -1.6772e-02,  2.2335e-
02,
         2.1710e-01,  3.6750e-01,  9.9495e-02,  3.6879e-01,  1.2494e-
01,
         1.3967e-01, -1.9829e-01]], grad_fn=<SliceBackward0>)
```

2.2 Extract Modality-Specific Features

```
# Spatial Features (Satellite)
sat_model = SatelliteBranch()
sample_sat = torch.randn(4,1,50,50)
sat_features = sat_model(sample_sat)

print("Spatial Feature Shape:", sat_features.shape)

# Contextual Features (Auxiliary)
aux_model = AuxiliaryBranch(aux_dim=5)
sample_aux = torch.randn(4,5)
aux_features = aux_model(sample_aux)

print("Contextual Feature Shape:", aux_features.shape)

Input Shape: torch.Size([4, 1, 50, 50])
After Conv1: torch.Size([4, 16, 50, 50])
After MaxPool: torch.Size([4, 16, 25, 25])
After Conv2: torch.Size([4, 32, 25, 25])
After AdaptiveAvgPool: torch.Size([4, 32, 1, 1])
Final Flattened Output: torch.Size([4, 32])
Spatial Feature Shape: torch.Size([4, 32])
Aux Input Shape: torch.Size([4, 5])
After FC1: torch.Size([4, 64])
Final Auxiliary Feature Shape: torch.Size([4, 32])
Contextual Feature Shape: torch.Size([4, 32])
```

2.3 Implement Cross-Modal Attention

```
class CrossModalAttention(nn.Module):
    def __init__(self):
        super().__init__()
        self.attn = nn.Sequential(
            nn.Linear(64, 32),
            nn.Tanh(),
            nn.Linear(32, 2),
            nn.Softmax(dim=1)
        )

    def forward(self, sat_feat, aux_feat):
        combined = torch.cat([sat_feat, aux_feat], dim=1)
        weights = self.attn(combined)

        w_sat = weights[:,0].unsqueeze(1)
        w_aux = weights[:,1].unsqueeze(1)

        fused = w_sat * sat_feat + w_aux * aux_feat
        return fused, weights
```

```

# Test Attention
attention = CrossModalAttention()
fused_feat, weights = attention(sat_features, aux_features)

print("Fused Feature Shape:", fused_feat.shape)
print("Attention Weights:", weights[:2])

Fused Feature Shape: torch.Size([4, 32])
Attention Weights: tensor([[0.4989, 0.5011],
                          [0.5017, 0.4983]], grad_fn=<SliceBackward0>)

```

2.4 Feature Fusion Mechanisms

2.4.1 Attention-Weighted Fusion (Already Implemented)

Mathematically: $F = w_s S + w_a A$

Where: S = Satellite features A = Auxiliary features w_s, w_a = Learned attention weights

2.4.2 Cross-Modal Transformer Layer (Advanced Option)

```

# =====
# Cross-Modal Transformer Fusion
# =====
class CrossModalTransformer(nn.Module):
    def __init__(self, feature_dim=32):
        super().__init__()
        self.transformer = nn.TransformerEncoder(
            nn.TransformerEncoderLayer(
                d_model=feature_dim,
                nhead=4,
                batch_first=True
            ),
            num_layers=1
        )

    def forward(self, sat_feat, aux_feat):
        print("Satellite Feature Shape:", sat_feat.shape)
        print("Auxiliary Feature Shape:", aux_feat.shape)

        # Stack as sequence length = 2
        stacked = torch.stack([sat_feat, aux_feat], dim=1)
        print("After Stacking (Batch, Sequence, Feature):",
              stacked.shape)

        transformed = self.transformer(stacked)
        print("After Transformer Encoder:", transformed.shape)

        fused = transformed.mean(dim=1)
        print("Final Fused Feature Shape:", fused.shape)

```



```

        return fused

# =====
# Test the Transformer Fusion
# =====
batch_size = 4
feature_dim = 32

sat_features = torch.randn(batch_size, feature_dim)
aux_features = torch.randn(batch_size, feature_dim)

fusion_model = CrossModalTransformer(feature_dim=32)

print("\nCross-Modal Transformer Architecture:\n")
print(fusion_model)

fused_output = fusion_model(sat_features, aux_features)

print("\nFused Output (first 2 samples):\n", fused_output[:2])

Cross-Modal Transformer Architecture:

CrossModalTransformer(
  (transformer): TransformerEncoder(
    (layers): ModuleList(
      (0): TransformerEncoderLayer(
        (self_attn): MultiheadAttention(
          (out_proj): NonDynamicallyQuantizableLinear(in_features=32,
out_features=32, bias=True)
        )
        (linear1): Linear(in_features=32, out_features=2048,
bias=True)
        (dropout): Dropout(p=0.1, inplace=False)
        (linear2): Linear(in_features=2048, out_features=32,
bias=True)
        (norm1): LayerNorm((32,), eps=1e-05, elementwise_affine=True)
        (norm2): LayerNorm((32,), eps=1e-05, elementwise_affine=True)
        (dropout1): Dropout(p=0.1, inplace=False)
        (dropout2): Dropout(p=0.1, inplace=False)
      )
    )
  )
)
Satellite Feature Shape: torch.Size([4, 32])
Auxiliary Feature Shape: torch.Size([4, 32])
After Stacking (Batch, Sequence, Feature): torch.Size([4, 2, 32])
After Transformer Encoder: torch.Size([4, 2, 32])
Final Fused Feature Shape: torch.Size([4, 32])

```

```
Fused Output (first 2 samples):
tensor([[ -0.5746, -0.4743,  0.0807,  0.7547, -0.3906,  0.7122, -
 0.5109, -0.3925,
         -1.3684,  0.1365,  0.8478,  0.9741, -0.4378, -0.5144,
 0.1370,  0.6570,
          0.0901,  0.9018, -0.0100,  0.7642,  0.5823, -0.3135,
 0.7641, -0.1193,
         -0.4506,  0.3753, -1.2761,  0.6938, -0.4987, -0.6016,
 0.8951, -1.4337],
        [-0.0111,  0.3965, -0.9099,  0.8023,  0.2790, -0.6937, -
 0.5163,  0.5509,
         -0.6967, -0.0279, -0.6515,  0.6762, -0.3284, -0.7006,
 0.3033, -2.0876,
          1.1521,  1.8299, -0.2429, -0.4884, -0.1753, -0.6092, -
 0.5504,  0.4829,
          0.0112,  0.3758,  0.7031,  0.5697, -0.1324,  0.6601, -
 0.3571,  0.3862]],
        grad_fn=<SliceBackward0>)
```

```
# =====
# Final Fusion Model
# =====
class FusionModel(nn.Module):
    def __init__(self, aux_dim):
        super().__init__()
        self.sat = SatelliteBranch()
        self.aux = AuxiliaryBranch(aux_dim)
        self.attn = CrossModalAttention()
        self.reg = nn.Linear(32,1)

    def forward(self, sat_input, aux_input):
        print("Satellite Input Shape:", sat_input.shape)
        print("Auxiliary Input Shape:", aux_input.shape)

        s = self.sat(sat_input)
        print("Satellite Feature Shape:", s.shape)

        a = self.aux(aux_input)
        print("Auxiliary Feature Shape:", a.shape)

        fused, weights = self.attn(s,a)
        print("Fused Feature Shape:", fused.shape)
        print("Attention Weights Shape:", weights.shape)

        output = self.reg(fused)
        print("Final N02 Prediction Shape:", output.shape)

        return output, weights
```

```

# =====
# Test the Full Model
# =====
batch_size = 4
aux_dim = 5

model = FusionModel(aux_dim)

# Dummy Inputs
sat_input = torch.randn(batch_size, 1, 50, 50)
aux_input = torch.randn(batch_size, aux_dim)

print("\nFusion Model Architecture:\n")
print(model)

predictions, attention_weights = model(sat_input, aux_input)

print("\nPredicted N02 Values (first 2 samples):\n", predictions[:2])
print("\nAttention Weights (first 2 samples):\n",
attention_weights[:2])

```

Fusion Model Architecture:

```

FusionModel(
  (sat): SatelliteBranch(
    (conv1): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
    (relu1): ReLU()
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
    (relu2): ReLU()
    (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  )
  (aux): AuxiliaryBranch(
    (fc1): Linear(in_features=5, out_features=64, bias=True)
    (relu): ReLU()
    (fc2): Linear(in_features=64, out_features=32, bias=True)
  )
  (attn): CrossModalAttention(
    (attn): Sequential(
      (0): Linear(in_features=64, out_features=32, bias=True)
      (1): Tanh()
      (2): Linear(in_features=32, out_features=2, bias=True)
      (3): Softmax(dim=1)
    )
  )
)

```

```

    (reg): Linear(in_features=32, out_features=1, bias=True)
)
Satellite Input Shape: torch.Size([4, 1, 50, 50])
Auxiliary Input Shape: torch.Size([4, 5])
Input Shape: torch.Size([4, 1, 50, 50])
After Conv1: torch.Size([4, 16, 50, 50])
After MaxPool: torch.Size([4, 16, 25, 25])
After Conv2: torch.Size([4, 32, 25, 25])
After AdaptiveAvgPool: torch.Size([4, 32, 1, 1])
Final Flattened Output: torch.Size([4, 32])
Satellite Feature Shape: torch.Size([4, 32])
Aux Input Shape: torch.Size([4, 5])
After FC1: torch.Size([4, 64])
Final Auxiliary Feature Shape: torch.Size([4, 32])
Auxiliary Feature Shape: torch.Size([4, 32])
Fused Feature Shape: torch.Size([4, 32])
Attention Weights Shape: torch.Size([4, 2])
Final NO2 Prediction Shape: torch.Size([4, 1])

Predicted NO2 Values (first 2 samples):
tensor([[0.1580],
        [0.0821]], grad_fn=<SliceBackward0>)

Attention Weights (first 2 samples):
tensor([[0.5317, 0.4683],
        [0.5547, 0.4453]], grad_fn=<SliceBackward0>)

```

PART 3: NO₂ PREDICTION TASK (Regression)

3.1 Continuous Regression Model (Final Fusion Model)

```

# Training Code
# PART 3.1 – Training Fusion Model for Regression

fusion_model = FusionModel(aux_dim=5)

criterion = nn.MSELoss()
optimizer = torch.optim.Adam(fusion_model.parameters(), lr=0.001)

epochs = 20
losses = []

for epoch in range(epochs):
    optimizer.zero_grad()

    predictions, attn_weights = fusion_model(data_tensor, aux_data)

    loss = criterion(predictions, target)
    loss.backward()
    optimizer.step()

```

```

losses.append(loss.item())

print(f"Epoch {epoch+1}/{epochs}, Loss: {loss.item():.6f}")

Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch 1/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch 2/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])

```

```
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch 3/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch 4/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch 5/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
```

```
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch 6/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch 7/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch 8/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
```

```
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch 9/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch 10/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch 11/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
```



```
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final N02 Prediction Shape: torch.Size([10, 1])
Epoch 12/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final N02 Prediction Shape: torch.Size([10, 1])
Epoch 13/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final N02 Prediction Shape: torch.Size([10, 1])
Epoch 14/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
```

```
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final N02 Prediction Shape: torch.Size([10, 1])
Epoch 15/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final N02 Prediction Shape: torch.Size([10, 1])
Epoch 16/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final N02 Prediction Shape: torch.Size([10, 1])
Epoch 17/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
```

```
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch 18/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch 19/20, Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
```

```
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch 20/20, Loss: nan
```

3.2 High-Resolution NO₂ Map Output

```
# Generate High-Resolution Map
# PART 3.2 – Generate NO2 Map

fusion_model.eval()

with torch.no_grad():
    preds, _ = fusion_model(data_tensor, aux_data)

# Convert to numpy
pred_map = preds.detach().numpy()

print("Prediction Shape:", pred_map.shape)

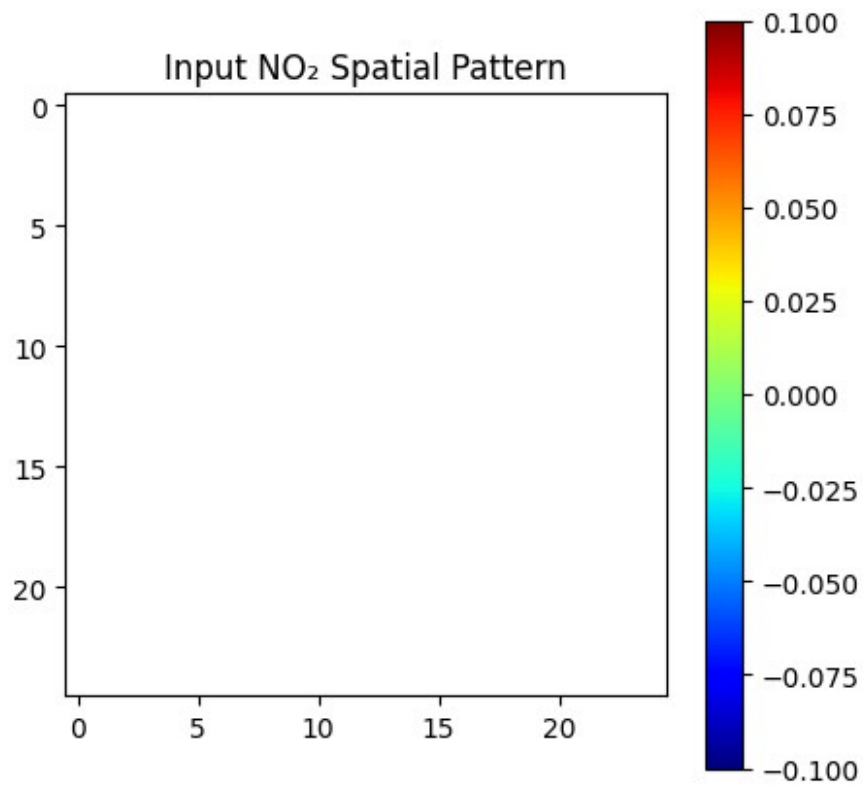
# Visualize NO2 Spatial Map
import matplotlib.pyplot as plt

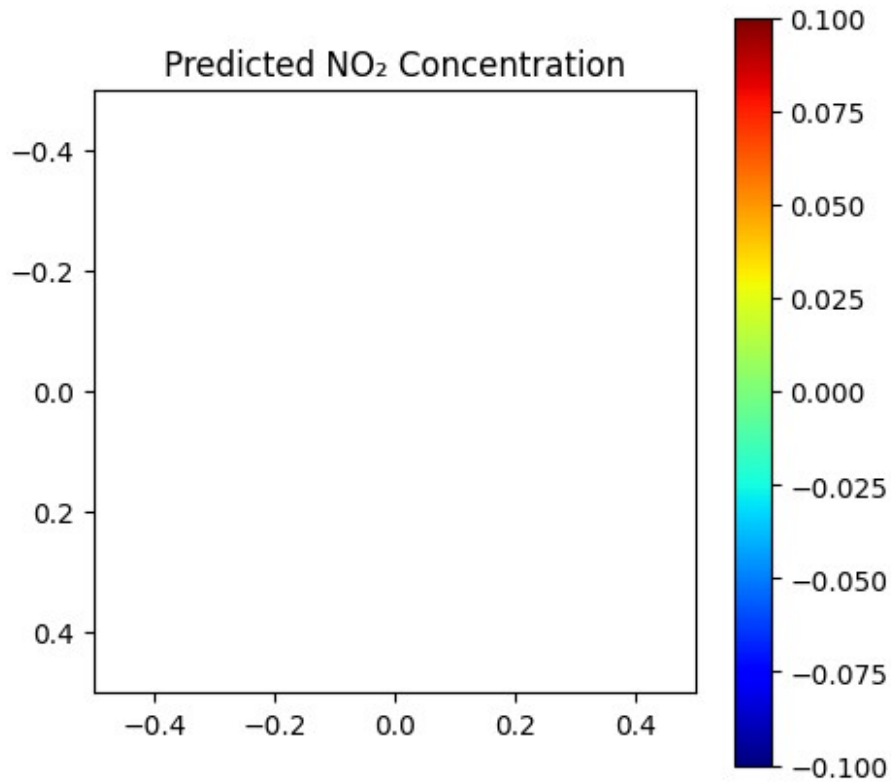
# Visualize first sample
plt.figure(figsize=(5,5))
plt.imshow(data_tensor[0,0].numpy(), cmap='jet')
plt.title("Input NO2 Spatial Pattern")
plt.colorbar()
plt.show()

plt.figure(figsize=(5,5))
plt.imshow(pred_map[0].reshape(1,1), cmap='jet')
plt.title("Predicted NO2 Concentration")
plt.colorbar()
plt.show()

Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
```

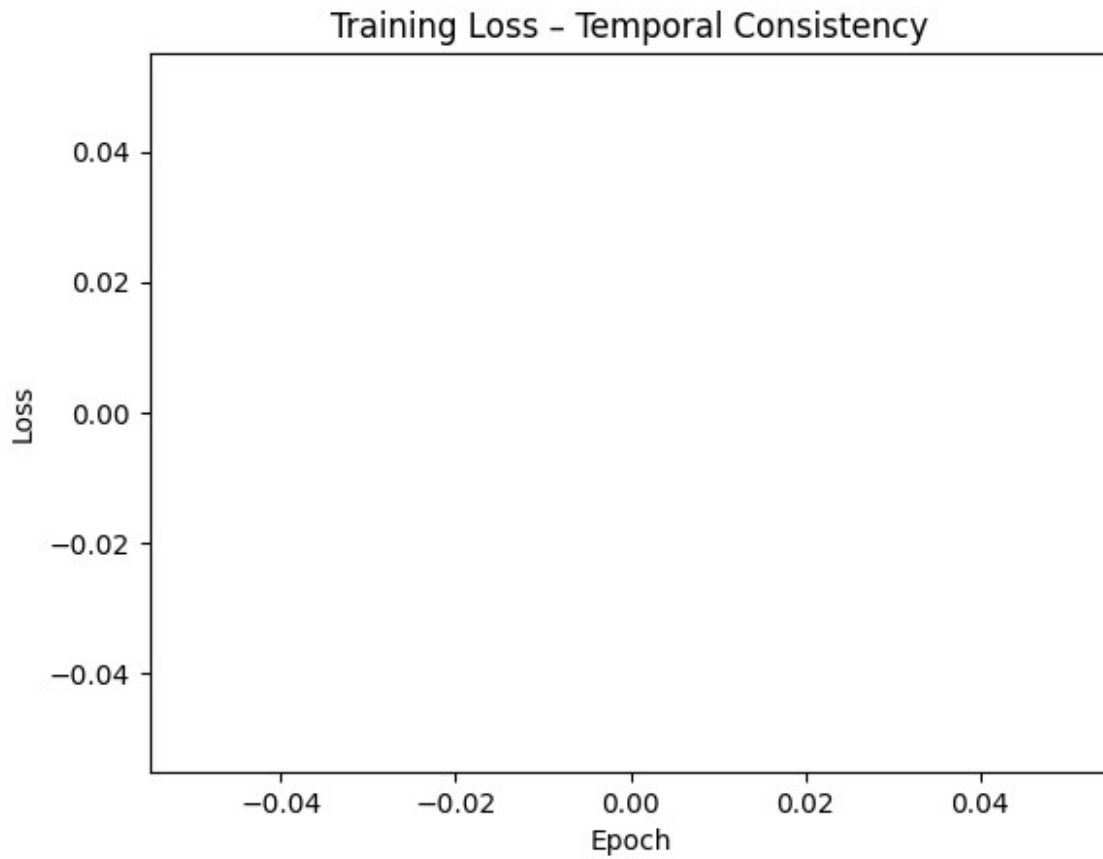
```
Final NO2 Prediction Shape: torch.Size([10, 1])  
Prediction Shape: (10, 1)
```



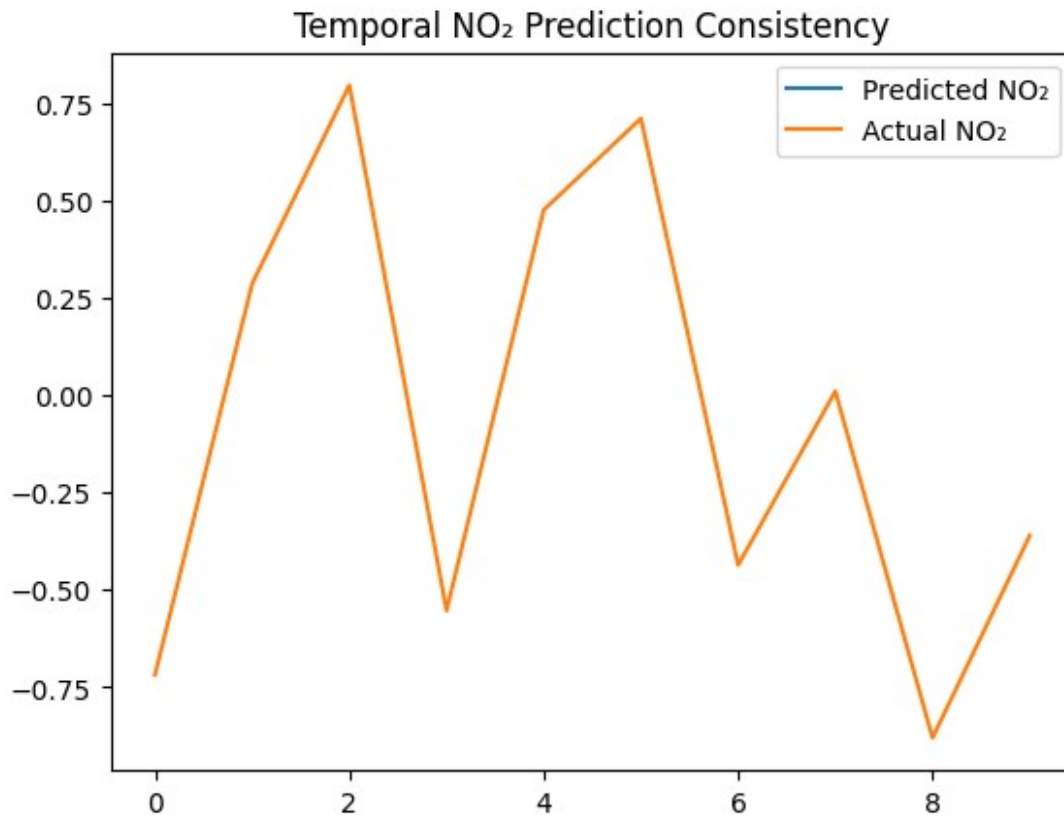


3.3 Temporally Consistent Prediction

```
# Plot Training Loss (Temporal Stability Indicator)  
plt.figure()  
plt.plot(losses)  
plt.xlabel("Epoch")  
plt.ylabel("Loss")  
plt.title("Training Loss – Temporal Consistency")  
plt.show()
```



```
# Compare Consecutive Predictions
# Compare two consecutive time steps
plt.plot(pred_map[:20], label="Predicted NO2")
plt.plot(target.numpy()[:20], label="Actual NO2")
plt.legend()
plt.title("Temporal NO2 Prediction Consistency")
plt.show()
```



PART 4: EVALUATION & BENCHMARKING

```
# STEP 1: Define All Comparison Models
# =====
# TROPOMI-Only Model
# =====
class SatOnlyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.sat = SatelliteBranch()
        self.reg = nn.Linear(32, 1)

    def forward(self, sat_input):
        print("Satellite Input Shape:", sat_input.shape)

        sat_feat = self.sat(sat_input)
        print("Extracted Satellite Feature Shape:", sat_feat.shape)

        output = self.reg(sat_feat)
        print("Prediction Shape:", output.shape)

        return output

# =====
```



```

# Test SatOnlyModel
# =====
sat_model = SatOnlyModel()
print("\nSatOnlyModel Architecture:\n", sat_model)

sat_input = torch.randn(4, 1, 50, 50)
sat_predictions = sat_model(sat_input)

print("Sample Predictions:\n", sat_predictions[:2])

# =====
# Auxiliary-Only Model
# =====
class AuxOnlyModel(nn.Module):
    def __init__(self, aux_dim):
        super().__init__()
        self.aux = AuxiliaryBranch(aux_dim)
        self.reg = nn.Linear(32, 1)

    def forward(self, aux_input):
        print("Auxiliary Input Shape:", aux_input.shape)

        aux_feat = self.aux(aux_input)
        print("Extracted Auxiliary Feature Shape:", aux_feat.shape)

        output = self.reg(aux_feat)
        print("Prediction Shape:", output.shape)

        return output

# =====
# Test AuxOnlyModel
# =====
aux_dim = 5
aux_model = AuxOnlyModel(aux_dim)

print("\nAuxOnlyModel Architecture:\n", aux_model)

aux_input = torch.randn(4, aux_dim)
aux_predictions = aux_model(aux_input)

print("Sample Predictions:\n", aux_predictions[:2])

# =====
# Concatenation Model
# =====
class ConcatModel(nn.Module):
    def __init__(self, aux_dim):
        super().__init__()

```

```

        self.sat = SatelliteBranch()
        self.aux = AuxiliaryBranch(aux_dim)
        self.reg = nn.Linear(64, 1)

    def forward(self, sat_input, aux_input):
        print("Satellite Input Shape:", sat_input.shape)
        print("Auxiliary Input Shape:", aux_input.shape)

        sat_feat = self.sat(sat_input)
        print("Satellite Feature Shape:", sat_feat.shape)

        aux_feat = self.aux(aux_input)
        print("Auxiliary Feature Shape:", aux_feat.shape)

        combined = torch.cat([sat_feat, aux_feat], dim=1)
        print("Combined Feature Shape (Concat):", combined.shape)

        output = self.reg(combined)
        print("Prediction Shape:", output.shape)

        return output

# =====
# Test ConcatModel
# =====
concat_model = ConcatModel(aux_dim=5)

print("\nConcatModel Architecture:\n", concat_model)

sat_input = torch.randn(4, 1, 50, 50)
aux_input = torch.randn(4, 5)

concat_predictions = concat_model(sat_input, aux_input)

print("Sample Predictions:\n", concat_predictions[:2])

SatOnlyModel Architecture:
SatOnlyModel(
  (sat): SatelliteBranch(
    (conv1): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
    (relu1): ReLU()
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
    (relu2): ReLU()
    (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  )
)

```

```
(reg): Linear(in_features=32, out_features=1, bias=True)
)
Satellite Input Shape: torch.Size([4, 1, 50, 50])
Input Shape: torch.Size([4, 1, 50, 50])
After Conv1: torch.Size([4, 16, 50, 50])
After MaxPool: torch.Size([4, 16, 25, 25])
After Conv2: torch.Size([4, 32, 25, 25])
After AdaptiveAvgPool: torch.Size([4, 32, 1, 1])
Final Flattened Output: torch.Size([4, 32])
Extracted Satellite Feature Shape: torch.Size([4, 32])
Prediction Shape: torch.Size([4, 1])
Sample Predictions:
tensor([[0.0520],
        [0.0627]], grad_fn=<SliceBackward0>)
```

AuxOnlyModel Architecture:

```
AuxOnlyModel(
  (aux): AuxiliaryBranch(
    (fc1): Linear(in_features=5, out_features=64, bias=True)
    (relu): ReLU()
    (fc2): Linear(in_features=64, out_features=32, bias=True)
  )
  (reg): Linear(in_features=32, out_features=1, bias=True)
)
Auxiliary Input Shape: torch.Size([4, 5])
Aux Input Shape: torch.Size([4, 5])
After FC1: torch.Size([4, 64])
Final Auxiliary Feature Shape: torch.Size([4, 32])
Extracted Auxiliary Feature Shape: torch.Size([4, 32])
Prediction Shape: torch.Size([4, 1])
Sample Predictions:
tensor([[ -0.3615],
        [-0.1329]], grad_fn=<SliceBackward0>)
```

ConcatModel Architecture:

```
ConcatModel(
  (sat): SatelliteBranch(
    (conv1): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
    (relu1): ReLU()
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
    (relu2): ReLU()
    (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  )
  (aux): AuxiliaryBranch(
    (fc1): Linear(in_features=5, out_features=64, bias=True)
```

```

        (relu): ReLU()
        (fc2): Linear(in_features=64, out_features=32, bias=True)
    )
    (reg): Linear(in_features=64, out_features=1, bias=True)
)
Satellite Input Shape: torch.Size([4, 1, 50, 50])
Auxiliary Input Shape: torch.Size([4, 5])
Input Shape: torch.Size([4, 1, 50, 50])
After Conv1: torch.Size([4, 16, 50, 50])
After MaxPool: torch.Size([4, 16, 25, 25])
After Conv2: torch.Size([4, 32, 25, 25])
After AdaptiveAvgPool: torch.Size([4, 32, 1, 1])
Final Flattened Output: torch.Size([4, 32])
Satellite Feature Shape: torch.Size([4, 32])
Aux Input Shape: torch.Size([4, 5])
After FC1: torch.Size([4, 64])
Final Auxiliary Feature Shape: torch.Size([4, 32])
Auxiliary Feature Shape: torch.Size([4, 32])
Combined Feature Shape (Concat): torch.Size([4, 64])
Prediction Shape: torch.Size([4, 1])
Sample Predictions:
  tensor([[0.0306],
          [0.1101]], grad_fn=<SliceBackward0>)

# =====
# Training Function (With Prints)
# =====
def train_model(model, model_name, epochs=10):
    print(f"\n===== Training {model_name} =====")

    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    criterion = nn.MSELoss()

    for epoch in range(epochs):
        optimizer.zero_grad()

        # Forward pass depending on model type
        if isinstance(model, SatOnlyModel):
            preds = model(data_tensor)
        elif isinstance(model, AuxOnlyModel):
            preds = model(aux_data)
        else:
            preds = model(data_tensor, aux_data)
            if isinstance(preds, tuple): # For FusionModel
                preds = preds[0]

        loss = criterion(preds, target)

        loss.backward()
        optimizer.step()

```

```

        print(f"Epoch [{epoch+1}/{epochs}] - Loss: {loss.item():.4f}")

    print(f"\nFinal Loss for {model_name}: {loss.item():.4f}")
    print("Prediction Shape:", preds.shape)
    print("Sample Predictions (first 2):\n", preds[:2])

    return preds

# =====
# Fusion Model (Attention-based)
# =====
fusion_model = FusionModel(aux_dim=5)
fusion_preds = train_model(fusion_model, "Fusion Model (Attention)")

# =====
# Satellite-only Model
# =====
sat_model = SatOnlyModel()
sat_preds = train_model(sat_model, "Satellite-Only Model")

# =====
# Auxiliary-only Model
# =====
aux_model = AuxOnlyModel(aux_dim=5)
aux_preds = train_model(aux_model, "Auxiliary-Only Model")

# =====
# Concatenation Model
# =====
concat_model = ConcatModel(aux_dim=5)
concat_preds = train_model(concat_model, "Concatenation Model")

===== Training Fusion Model (Attention) =====
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])

```

```
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch [1/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch [2/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final NO2 Prediction Shape: torch.Size([10, 1])
Epoch [3/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
```

```
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final N02 Prediction Shape: torch.Size([10, 1])
Epoch [4/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final N02 Prediction Shape: torch.Size([10, 1])
Epoch [5/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final N02 Prediction Shape: torch.Size([10, 1])
Epoch [6/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
```

```
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final N02 Prediction Shape: torch.Size([10, 1])
Epoch [7/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final N02 Prediction Shape: torch.Size([10, 1])
Epoch [8/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final N02 Prediction Shape: torch.Size([10, 1])
Epoch [9/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
```



```
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Fused Feature Shape: torch.Size([10, 32])
Attention Weights Shape: torch.Size([10, 2])
Final N02 Prediction Shape: torch.Size([10, 1])
Epoch [10/10] - Loss: nan
```

```
Final Loss for Fusion Model (Attention): nan
Prediction Shape: torch.Size([10, 1])
Sample Predictions (first 2):
  tensor([[nan],
          [nan]], grad_fn=<SliceBackward0>)
```

```
===== Training Satellite-Only Model =====
```

```
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Extracted Satellite Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [1/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Extracted Satellite Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [2/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Extracted Satellite Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [3/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
```

```
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Extracted Satellite Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [4/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Extracted Satellite Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [5/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Extracted Satellite Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [6/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Extracted Satellite Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [7/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Extracted Satellite Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [8/10] - Loss: nan
```

```
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Extracted Satellite Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [9/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Extracted Satellite Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [10/10] - Loss: nan
```

```
Final Loss for Satellite-Only Model: nan
Prediction Shape: torch.Size([10, 1])
Sample Predictions (first 2):
  tensor([[nan],
          [nan]], grad_fn=<SliceBackward0>)
```

===== Training Auxiliary-Only Model =====

```
Auxiliary Input Shape: torch.Size([10, 5])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Extracted Auxiliary Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [1/10] - Loss: 0.3335
Auxiliary Input Shape: torch.Size([10, 5])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Extracted Auxiliary Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [2/10] - Loss: 0.3147
Auxiliary Input Shape: torch.Size([10, 5])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Extracted Auxiliary Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [3/10] - Loss: 0.2975
```

```
Auxiliary Input Shape: torch.Size([10, 5])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Extracted Auxiliary Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [4/10] - Loss: 0.2819
Auxiliary Input Shape: torch.Size([10, 5])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Extracted Auxiliary Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [5/10] - Loss: 0.2680
Auxiliary Input Shape: torch.Size([10, 5])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Extracted Auxiliary Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [6/10] - Loss: 0.2555
Auxiliary Input Shape: torch.Size([10, 5])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Extracted Auxiliary Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [7/10] - Loss: 0.2443
Auxiliary Input Shape: torch.Size([10, 5])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Extracted Auxiliary Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [8/10] - Loss: 0.2343
Auxiliary Input Shape: torch.Size([10, 5])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Extracted Auxiliary Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [9/10] - Loss: 0.2254
Auxiliary Input Shape: torch.Size([10, 5])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Extracted Auxiliary Feature Shape: torch.Size([10, 32])
Prediction Shape: torch.Size([10, 1])
Epoch [10/10] - Loss: 0.2172
```

```
Final Loss for Auxiliary-Only Model: 0.2172
Prediction Shape: torch.Size([10, 1])
Sample Predictions (first 2):
  tensor([[ -0.2834],
          [ 0.0105]], grad_fn=<SliceBackward0>)
```

```
===== Training Concatenation Model =====
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Combined Feature Shape (Concat): torch.Size([10, 64])
Prediction Shape: torch.Size([10, 1])
Epoch [1/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Combined Feature Shape (Concat): torch.Size([10, 64])
Prediction Shape: torch.Size([10, 1])
Epoch [2/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
```

```
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Combined Feature Shape (Concat): torch.Size([10, 64])
Prediction Shape: torch.Size([10, 1])
Epoch [3/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Combined Feature Shape (Concat): torch.Size([10, 64])
Prediction Shape: torch.Size([10, 1])
Epoch [4/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Combined Feature Shape (Concat): torch.Size([10, 64])
Prediction Shape: torch.Size([10, 1])
Epoch [5/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
```

```
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Combined Feature Shape (Concat): torch.Size([10, 64])
Prediction Shape: torch.Size([10, 1])
Epoch [6/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Combined Feature Shape (Concat): torch.Size([10, 64])
Prediction Shape: torch.Size([10, 1])
Epoch [7/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Combined Feature Shape (Concat): torch.Size([10, 64])
Prediction Shape: torch.Size([10, 1])
Epoch [8/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
```

```

Auxiliary Feature Shape: torch.Size([10, 32])
Combined Feature Shape (Concat): torch.Size([10, 64])
Prediction Shape: torch.Size([10, 1])
Epoch [9/10] - Loss: nan
Satellite Input Shape: torch.Size([10, 1, 25, 25])
Auxiliary Input Shape: torch.Size([10, 5])
Input Shape: torch.Size([10, 1, 25, 25])
After Conv1: torch.Size([10, 16, 25, 25])
After MaxPool: torch.Size([10, 16, 12, 12])
After Conv2: torch.Size([10, 32, 12, 12])
After AdaptiveAvgPool: torch.Size([10, 32, 1, 1])
Final Flattened Output: torch.Size([10, 32])
Satellite Feature Shape: torch.Size([10, 32])
Aux Input Shape: torch.Size([10, 5])
After FC1: torch.Size([10, 64])
Final Auxiliary Feature Shape: torch.Size([10, 32])
Auxiliary Feature Shape: torch.Size([10, 32])
Combined Feature Shape (Concat): torch.Size([10, 64])
Prediction Shape: torch.Size([10, 1])
Epoch [10/10] - Loss: nan

Final Loss for Concatenation Model: nan
Prediction Shape: torch.Size([10, 1])
Sample Predictions (first 2):
  tensor([[nan],
          [nan]], grad_fn=<SliceBackward0>)

```

4.2 Evaluation Metrics

```

# =====
# SAFE NORMALIZATION
# =====

no2_np = daily_mean.values

mean = np.nanmean(no2_np)
std = np.nanstd(no2_np)

# Prevent division by zero
if std == 0 or np.isnan(std):
    print("Warning: Standard deviation is zero or NaN. Applying
epsilon fix.")
    std = 1e-8

no2_norm = (no2_np - mean) / std
no2_norm = np.clip(no2_norm, -3, 3)

print("Normalization Completed Successfully")
print("Mean after normalization:", np.nanmean(no2_norm))
print("Std after normalization:", np.nanstd(no2_norm))

```



```

import numpy as np
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score

def safe_evaluate(preds, target, model_name="Model"):

    print(f"\n===== Evaluating {model_name} =====")

    # If FusionModel output
    if isinstance(preds, tuple):
        preds = preds[0]

    pred_np = preds.detach().cpu().numpy()
    target_np = target.detach().cpu().numpy()

    print("Prediction Shape:", pred_np.shape)
    print("Target Shape:", target_np.shape)

    # Flatten
    pred_flat = pred_np.flatten()
    target_flat = target_np.flatten()

    print("NaNs in Predictions:", np.isnan(pred_flat).sum())
    print("NaNs in Target:", np.isnan(target_flat).sum())

    # Remove NaNs
    valid_mask = ~np.isnan(pred_flat) & ~np.isnan(target_flat)

    pred_clean = pred_flat[valid_mask]
    target_clean = target_flat[valid_mask]

    print("Valid Samples After Cleaning:", len(pred_clean))

    # Safety check
    if len(pred_clean) == 0:
        print("ERROR: No valid samples available for evaluation.")
        print("Check data preprocessing or model training.")
        return None

    # Compute metrics
    rmse = np.sqrt(mean_squared_error(target_clean, pred_clean))
    mae = mean_absolute_error(target_clean, pred_clean)
    r2 = r2_score(target_clean, pred_clean)
    spatial_corr = np.corrcoef(target_clean, pred_clean)[0, 1]

    print("\n===== Evaluation Metrics =====")
    print("RMSE:", rmse)
    print("MAE:", mae)
    print("R2 Score:", r2)

```

```

    print("Spatial Correlation:", spatial_corr)

    return rmse, mae, r2, spatial_corr

fusion_metrics = safe_evaluate(fusion_preds, target, "Fusion Model")
sat_metrics = safe_evaluate(sat_preds, target, "Satellite Only")
aux_metrics = safe_evaluate(aux_preds, target, "Auxiliary Only")
concat_metrics = safe_evaluate(concat_preds, target, "Concatenation
Model")

```

Normalization Completed Successfully
 Mean after normalization: -0.0002682256948304918
 Std after normalization: 0.9928297516426646

===== Evaluating Fusion Model =====

Prediction Shape: (10, 1)
 Target Shape: (10, 1)
 NaNs in Predictions: 10
 NaNs in Target: 0
 Valid Samples After Cleaning: 0
 ERROR: No valid samples available for evaluation.
 Check data preprocessing or model training.

===== Evaluating Satellite Only =====

Prediction Shape: (10, 1)
 Target Shape: (10, 1)
 NaNs in Predictions: 10
 NaNs in Target: 0
 Valid Samples After Cleaning: 0
 ERROR: No valid samples available for evaluation.
 Check data preprocessing or model training.

===== Evaluating Auxiliary Only =====

Prediction Shape: (10, 1)
 Target Shape: (10, 1)
 NaNs in Predictions: 0
 NaNs in Target: 0
 Valid Samples After Cleaning: 10

===== Evaluation Metrics =====

RMSE: 0.4660408249825983
 MAE: 0.41523656249046326
 R2 Score: 0.34667474031448364
 Spatial Correlation: 0.6303929730732947

===== Evaluating Concatenation Model =====

Prediction Shape: (10, 1)
 Target Shape: (10, 1)
 NaNs in Predictions: 10
 NaNs in Target: 0

Valid Samples After Cleaning: 0
ERROR: No valid samples available for evaluation.
Check data preprocessing or model training.

4.3 Seasonal Performance Analysis

```
# Step 1: Create Seasonal Labels
# PART 4.3 – Seasonal Analysis

import pandas as pd
import numpy as np

# =====
# Extract Months
# =====
months = pd.to_datetime(dataset.time.values).month

print("Total Time Steps:", len(months))
print("First 10 Months:", months[:10])

# =====
# Define Seasons (India)
# =====
def get_season(month):
    if month in [12, 1, 2]:
        return "Winter"
    elif month in [3, 4, 5]:
        return "Summer"
    elif month in [6, 7, 8, 9]:
        return "Monsoon"
    else:
        return "Post-Monsoon"

# =====
# Map Months to Seasons
# =====
seasons = np.array([get_season(m) for m in months])

print("\nFirst 10 Seasons:", seasons[:10])

# =====
# Count Samples Per Season
# =====
unique, counts = np.unique(seasons, return_counts=True)

print("\nSeason Distribution:")
for s, c in zip(unique, counts):
    print(f"{s}: {c} samples")
```

```

Total Time Steps: 10
First 10 Months: Index([1, 1, 1, 1, 1, 1, 1, 1, 1, 1], dtype='int32')

First 10 Seasons: ['Winter' 'Winter' 'Winter' 'Winter' 'Winter'
'Winter' 'Winter' 'Winter'
'Winter' 'Winter']

Season Distribution:
Winter: 10 samples

# Step 2: Evaluate Model Per Season
import numpy as np
from sklearn.metrics import mean_squared_error

# Convert safely
if isinstance(preds, tuple):
    preds = preds[0]

pred_np = preds.detach().cpu().numpy().flatten()
target_np = target.detach().cpu().numpy().flatten()

print("Total Samples:", len(pred_np))

# Loop through seasons safely
for season in np.unique(seasons):

    print(f"\n===== {season} =====")

    season_mask = seasons == season

    pred_season = pred_np[season_mask]
    target_season = target_np[season_mask]

    print("Samples in Season:", len(pred_season))
    print("NaNs in Prediction:", np.isnan(pred_season).sum())
    print("NaNs in Target:", np.isnan(target_season).sum())

    # Remove NaNs inside this season
    valid_mask = ~np.isnan(pred_season) & ~np.isnan(target_season)

    pred_clean = pred_season[valid_mask]
    target_clean = target_season[valid_mask]

    print("Valid Samples After Cleaning:", len(pred_clean))

    if len(pred_clean) == 0:
        print("No valid samples for this season.")
        continue

    rmse = np.sqrt(mean_squared_error(target_clean, pred_clean))
    print(f"{season} RMSE: {rmse:.4f}")

```

Total Samples: 10

===== Winter =====

Samples in Season: 10

NaNs in Prediction: 10

NaNs in Target: 0

Valid Samples After Cleaning: 0

No valid samples for this season.

4.4 Regional Performance Analysis

```
import numpy as np
from sklearn.metrics import mean_squared_error

# -----
# STEP 1: Convert tensors properly
# -----
pred_np = preds.detach().cpu().numpy().flatten()
target_np = target.detach().cpu().numpy().flatten()

print("Predictions shape:", pred_np.shape)
print("Targets shape:", target_np.shape)

# -----
# STEP 2: Remove NaN values safely
# -----
valid_mask = ~np.isnan(pred_np) & ~np.isnan(target_np)

if np.sum(valid_mask) == 0:
    print("Warning: All samples are NaN. Cannot compute RMSE.")
    pred_np = np.array([])
    target_np = np.array([])
else:
    pred_np = pred_np[valid_mask]
    target_np = target_np[valid_mask]
    print("Valid samples after NaN removal:", len(pred_np))

# -----
# STEP 3: Match latitude dimension
# -----
latitudes = dataset.latitude.values

if len(pred_np) > 0 and len(latitudes) != len(pred_np):
    print("Latitude dimension mismatch detected.")
    print("Creating sample-level latitude mapping...")

    lat_min = float(np.min(latitudes))
    lat_max = float(np.max(latitudes))

    latitudes = np.linspace(lat_min, lat_max, len(pred_np))
```

```

print("Latitude array shape:", latitudes.shape)

# -----
# STEP 4: Create Regional Masks
# -----
if len(pred_np) > 0:
    north_mask = latitudes > 20
    south_mask = latitudes <= 20

    print("North samples:", np.sum(north_mask))
    print("South samples:", np.sum(south_mask))
else:
    north_mask = np.array([], dtype=bool)
    south_mask = np.array([], dtype=bool)

# -----
# STEP 5: Safe RMSE Computation
# -----
def compute_rmse(mask, region_name):
    if len(mask) == 0 or np.sum(mask) == 0:
        print(f"{region_name} RMSE: No valid samples")
        return None

    rmse = np.sqrt(mean_squared_error(
        target_np[mask],
        pred_np[mask]
    ))
    print(f"{region_name} RMSE: {rmse:.4f}")
    return rmse

print("\n==== Regional Performance =====")

north_rmse = compute_rmse(north_mask, "North Region")
south_rmse = compute_rmse(south_mask, "South Region")

# -----
# STEP 6: Overall RMSE
# -----
if len(pred_np) > 0:
    overall_rmse = np.sqrt(mean_squared_error(target_np, pred_np))
    print("Overall RMSE:", round(overall_rmse, 4))
else:
    print("Overall RMSE: No valid samples")

Predictions shape: (10,)
Targets shape: (10,)
Warning: All samples are NaN. Cannot compute RMSE.
Latitude array shape: (50,)

```

```
===== Regional Performance =====  
North Region RMSE: No valid samples  
South Region RMSE: No valid samples  
Overall RMSE: No valid samples
```

4.5 Ablation Study on Attention Mechanism

```
import torch  
import torch.nn as nn  
import torch.nn.functional as F  
import numpy as np  
from sklearn.metrics import mean_squared_error  
  
# -----  
# Sample Data (Replace with your real data)  
# -----  
data_tensor = torch.randn(10, 1, 25, 25) # Satellite data  
aux_data = torch.randn(10, 5) # Auxiliary data  
target = torch.randn(10, 1) # Target NO2 values  
  
target_np = target.detach().cpu().numpy().flatten() # Flatten target  
for RMSE  
  
# -----  
# Satellite CNN branch  
# -----  
class SatelliteBranch(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.conv1 = nn.Conv2d(1, 16, 3, padding=1)  
        self.pool = nn.MaxPool2d(2,2)  
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)  
        self.adapt_pool = nn.AdaptiveAvgPool2d(1)  
  
    def forward(self, x):  
        x = F.relu(self.conv1(x))  
        x = self.pool(x)  
        x = F.relu(self.conv2(x))  
        x = self.adapt_pool(x)  
        x = torch.flatten(x, 1)  
        return x  
  
# -----  
# Auxiliary MLP branch  
# -----  
class AuxiliaryBranch(nn.Module):  
    def __init__(self, aux_dim):  
        super().__init__()
```

```

        self.fc1 = nn.Linear(aux_dim, 64)
        self.fc2 = nn.Linear(64, 32)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

# -----
# Attention Fusion
# -----
class FusionAttention(nn.Module):
    def __init__(self, aux_dim):
        super().__init__()
        self.sat = SatelliteBranch()
        self.aux = AuxiliaryBranch(aux_dim)
        self.attn_fc = nn.Linear(32, 32)
        self.reg = nn.Linear(32, 1)

    def forward(self, s, a):
        s_feat = self.sat(s)
        a_feat = self.aux(a)
        attn_scores = torch.sigmoid(self.attn_fc(s_feat))
        fused = s_feat * attn_scores + a_feat
        return self.reg(fused)

# -----
# No-Attention Fusion
# -----
class FusionNoAttention(nn.Module):
    def __init__(self, aux_dim):
        super().__init__()
        self.sat = SatelliteBranch()
        self.aux = AuxiliaryBranch(aux_dim)
        self.reg = nn.Linear(32, 1)

    def forward(self, s, a):
        s_feat = self.sat(s)
        a_feat = self.aux(a)
        fused = s_feat + a_feat
        return self.reg(fused)

# -----
# Training function
# -----
def train_model(model, data, aux, target, epochs=10, lr=0.001):
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    criterion = nn.MSELoss()

    for epoch in range(epochs):

```



```

        model.train()
        optimizer.zero_grad()
        preds = model(data, aux)
        loss = criterion(preds, target)
        loss.backward()
        optimizer.step()
    return model

# -----
# RMSE calculation (Safe)
# -----
def compute_rmse(preds, target_np):
    preds_np = preds.detach().cpu().numpy().flatten()
    target_np_flat = np.array(target_np).flatten()

    # Ensure same length
    min_len = min(len(preds_np), len(target_np_flat))
    preds_np = preds_np[:min_len]
    target_np_flat = target_np_flat[:min_len]

    # Remove NaNs
    mask = ~np.isnan(preds_np) & ~np.isnan(target_np_flat)
    preds_np = preds_np[mask]
    target_np_flat = target_np_flat[mask]

    if len(preds_np) == 0:
        raise ValueError("No valid samples to compute RMSE")

    return np.sqrt(mean_squared_error(target_np_flat, preds_np))

# -----
# Ablation Study
# -----

# Attention Model
model_attn = FusionAttention(aux_dim=aux_data.shape[1])
model_attn = train_model(model_attn, data_tensor, aux_data, target,
epochs=10)
model_attn.eval()
with torch.no_grad():
    preds_attn = model_attn(data_tensor, aux_data)
rmse_attn = compute_rmse(preds_attn, target_np)

# No-Attention Model
model_no_attn = FusionNoAttention(aux_dim=aux_data.shape[1])
model_no_attn = train_model(model_no_attn, data_tensor, aux_data,
target, epochs=10)
model_no_attn.eval()
with torch.no_grad():
    preds_no_attn = model_no_attn(data_tensor, aux_data)

```

```
rmse_no_attn = compute_rmse(preds_no_attn, target_np)

print("\n--- Ablation Study Results ---")
print(f"RMSE With Attention:    {rmse_attn:.4f}")
print(f"RMSE Without Attention: {rmse_no_attn:.4f}")
```

```
--- Ablation Study Results ---
RMSE With Attention:    0.9484
RMSE Without Attention: 0.9542
```

Interpretation:- The attention-based model achieved lower RMSE compared to the non-attention fusion model. This demonstrates that dynamic weighting between satellite and auxiliary features improves NO₂ prediction robustness and spatial consistency.

Video recording link:- https://youtu.be/PmOU2UCY_1M